

Article

A Novel High-Precision Railway Obstacle Detection Algorithm Based on 3D LiDAR

Zongliang Nan ^{1,2}, Guoan Zhu ^{1,2}, Xu Zhang ³, Xuechun Lin ^{1,2} and Yingying Yang ^{1,2,*}¹ Laboratory of All-Solid-State Light Sources, Institute of Semiconductors, Chinese Academy of Sciences, Beijing 100083, China; nanzongliang@semi.ac.cn (Z.N.); zhuga@semi.ac.cn (G.Z.); xclin@semi.ac.cn (X.L.)² College of Materials Science and Opto-Electronic Technology, University of Chinese Academy of Sciences, Beijing 101407, China³ Shenghong (Taizhou) Laser Technology Co., Ltd., Taizhou 318001, China; zhangxuxu@semi.ac.cn

* Correspondence: yanggy@semi.ac.cn

Abstract: This article presents a high-precision obstacle detection algorithm using 3D mechanical LiDAR to meet railway safety requirements. To address the potential errors in the point cloud, we propose a calibration method based on projection and a novel rail extraction algorithm that effectively handles terrain variations and preserves the point cloud characteristics of the track area. We address the limitations of the traditional process involving fixed Euclidean thresholds by proposing a modulation function based on directional density variations to adjust the threshold dynamically. Finally, using PCA and local-ICP, we conduct feature analysis and classification of the clustered data to obtain the obstacle clusters. We conducted continuous experiments on the testing site, and the results showed that our system and algorithm achieved an STDR (stable detection rate) of over 95% for obstacles with a size of 15 cm × 15 cm × 15 cm in the range of ±25 m; at the same time, for obstacles of 10 cm × 10 cm × 10 cm, an STDR of over 80% was achieved within a range of ±20 m. This research provides a possible solution and approach for railway security via obstacle detection.

Keywords: LiDAR; railway; SFRE; PCA; local-ICP

Citation: Nan, Z.; Zhu, G.; Zhang, X.; Lin, X.; Yang, Y. A Novel High-Precision Railway Obstacle Detection Algorithm Based on 3D LiDAR. *Sensors* **2024**, *24*, 3148. <https://doi.org/10.3390/s24103148>

Academic Editor: Shuo Shi

Received: 9 April 2024

Revised: 10 May 2024

Accepted: 10 May 2024

Published: 15 May 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

With the rapid advancements in rail transportation in recent years [1,2], integrating fully automated operation modes into railway systems is becoming more prevalent [3]. This necessitates strengthening risk prevention and control measures to avoid collisions with obstacles during rail operations [4]. As a result, there is an urgent demand for detecting potential obstacles along railway tracks, such as pedestrians, animals, and falling rocks. LiDAR technology has emerged as a crucial component in detection compared to traditional visual sensors due to its exceptional obstacle-detection capabilities, precise detection accuracy, and adaptability to different environmental conditions [5,6]. LiDAR collects echo signals from obstacles by utilizing laser pulses, generating a comprehensive three-dimensional point cloud output that provides detailed information about surrounding objects [7,8].

In order to achieve higher detection accuracy, researchers have developed various obstacle-recognition methods specifically designed for point clouds. For instance, deep learning networks such as PointNet [9] and PointNet++ [10] have been introduced for the processing of complex point clouds. These networks utilize fully connected layers to perform tasks like segmentation and classification in intricate scenarios. At the same time, LiDAR obstacle detection algorithms based on CNN networks have been developed and applied [11]. However, machine learning-based methods require many samples and regular updates to the model weights, limiting the timeliness of their application [12].

In contrast, previous non-machine learning algorithms relied on the assumption of flat road surfaces for obstacle identification [13]. For example, Alberto et al. [14] proposed using elevation thresholds derived from consecutive laser returns to differentiate curbside obstacles. Recognizing the limitations of this flat road assumption, Asvadi et al. [15] employed a segmented approach. They utilized the Random Sample Consensus (RANSAC) algorithm for ground point segmentation under various slope conditions and leveraged a voxel grid model to discern stationary and moving obstacle point clouds.

Additionally, other famous techniques have been applied to processing obstacles in point clouds. These include density-based algorithms such as k-means clustering [16], Euclidean clustering [17], and density-based spatial clustering (DSC) [18]. These algorithms aim to group points within a certain threshold around a central point into clusters, which are then used to detect obstacles based on cluster characteristics. However, traditional methods often utilize a fixed neighborhood threshold for the entire point cloud scene. This approach faces limitations because LiDAR point clouds are typically unevenly distributed, with significant density variations.

In addressing this limitation, Gao et al. [19] proposed a dynamic threshold DSC algorithm that overcomes this challenge. Their approach leverages an elliptical model to characterize the local environment and dynamically adapt the neighborhood radius based on the central point's position. This innovative technique leads to enhanced clustering algorithm performance. It has been successfully applied in obstacle avoidance experiments using onboard LiDAR systems. Jiang et al. [20] enhanced the clustering performance of obstacle clusters by introducing a modulation technique that adjusts the clustering radius based on the horizontal resolution θ and pitch resolution ω of the LiDAR. This modulation enables precise radius adjustments at various locations.

Numerous algorithms using point clouds have been employed in the obstacle detection (OD) process. For instance, Xie et al. [21] successfully detected moving obstacles by integrating a dynamic point-tracking model with a Kalman filter. This approach effectively captures and tracks objects in motion. Similarly, Frank et al. [22] implemented moving object detection in point cloud streaming input by combining the iterative closest point (ICP) algorithm with a Kalman filter, leveraging local convexity criteria.

Before conducting our research, we investigated the application of different types of LiDAR in specific projects. LiDAR has gradually been applied to railway projects, but different applications, such as rail extraction and intersection recognition, rely on different sensor models for the pipeline of the algorithm [23]. B. Borgmann et al. used a Velodyne HDL-64E LiDAR and proposed a height threshold method to achieve segmentation of ground point clouds; the method was based on an implicit shape model (ISM) and successfully achieved detection of person point clouds, but the study did not mention detection of smaller obstacles [24]. P. Burger et al. utilized the scanning characteristics of Velodyne sensors to achieve fast cluster segmentation in off-road environments by labeling discontinuous fixed points. However, this approach relies on Velodyne sensors and is suitable for dynamic point clouds [25]. Current point cloud OD algorithms face challenges such as limited accuracy and suitability in basic scenarios [26]. To overcome these limitations and cater to the obstacle detection requirements in railway applications, we have devised a high-precision OD algorithm using a mechanical 3D LiDAR. Our algorithm offers a comprehensive solution that accurately identifies obstacles in railway environments. We conducted extensive experiments and continuous testing of the developed algorithm within our simulated railway test site, effectively showcasing its remarkable effectiveness and robustness.

This article presents a comprehensive algorithm for railway obstacle detection, as demonstrated in Figure 1. Our contributions are as follows:

- (a) We propose a novel method for rail extraction based on LiDAR scanline features. This method overcomes the shortcomings of traditional ground segmentation algorithms in the segmentation process (to effectively filter the point cloud of raised obstacles on

the ground and apply the algorithm to less strictly flat road surfaces) and accurately preserves the point cloud information of the track region.

- (b) We have addressed the fixed threshold limitation in traditional Euclidean clustering algorithms and proposed an adaptive algorithm with tunable thresholds.
- (c) The proposed algorithm achieves an STDR of 96% for obstacles of $15\text{ cm} \times 15\text{ cm} \times 15\text{ cm}$ within a range of $\pm 25\text{ m}$ and of 84% for obstacles of $10\text{ cm} \times 10\text{ cm} \times 10\text{ cm}$ within a range of $\pm 20\text{ m}$. We conducted diverse and repeated experiments in a simulated railway environment, yielding satisfactory results and demonstrating the potential for large-scale application.

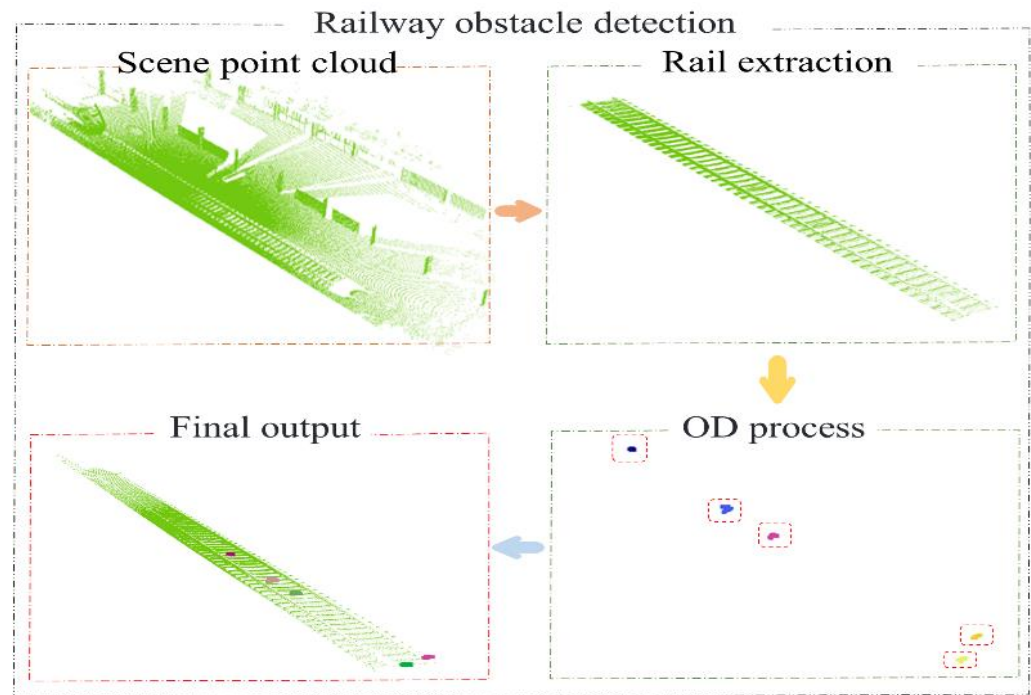


Figure 1. The operational steps of our railway OD algorithm.

The starting point of our system is to detect and prevent some dangerous factors (such as falling rocks, mudslides, etc.), and these obstacles invading the track area will affect driving safety. Therefore, our system can be installed in fixed areas with high risk factors to achieve early warning of threatening obstacles.

The workflow of this article is as follows: In Section 2, we introduce the scanning principle of the equipment we used. In Section 3, we introduce the key processes and steps of the algorithm, and we further improve the traditional Euclidean clustering in this study. In Section 4, we conduct performance and robustness testing of the algorithm in the experimental field, and the results demonstrated satisfactory accuracy of the algorithm's OD performance. Finally, we conclude the article with a summary and outlook.

2. Scanning Mechanism of Mechanical LiDAR

The 3D LiDAR depicted in Figure 2 is utilized within our study. While the system's autonomous development falls outside this article's scope, our primary focus is elucidating its scanning mechanism. Our device completes the acquisition of a single scanning line in the horizontal direction with a frequency f . The pulse points are distributed according to a fixed lateral angle resolution, and high-precision servo motors are used to control the pitch direction. The scanning line is distributed according to a fixed pitch angle resolution, which is used to obtain three-dimensional point cloud data for the rail area within the range of $\pm 25\text{ m}$. We control the maximum error of scanning for the same point

to be below 2 cm. Due to our scanning mechanism, we obtained a point cloud map with a concentrated distance between points in the central region and a dispersed distance between points in the edge region (Figure 2b).

We exercise control over the precision of the point cloud map by adjusting the scanning time T and the pitch scanning range $[\omega_{start}, \omega_{stop}]$. The following formula is used to calculate the parameters:

$$k = T \times f \quad (1)$$

$$\phi = \omega_{stop} - \omega_{start} \quad (2)$$

$$\delta_{pitch} = \phi / k \quad (3)$$

In Equations (1)–(3), k represents the number of scanning lines in a single frame of the point cloud map, T represents the scanning time, $\delta_{horizontal}$ represents the known horizontal angular resolution, and δ_{pitch} signifies the pitch angular resolution.

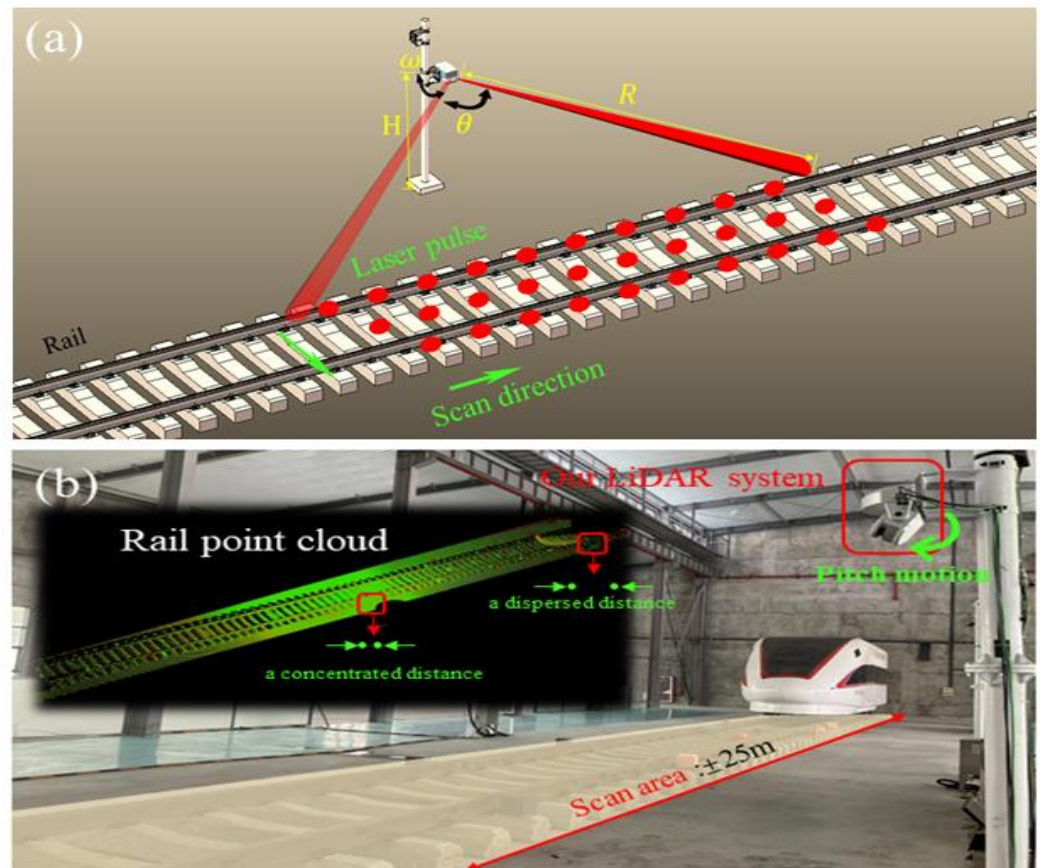


Figure 2. Installation method and scanning mechanism of the 3D mechanical LiDAR. (a) is a schematic diagram that we use to describe the scanning method of the device; (b) is a schematic diagram of installing our equipment on site, with the equipment installed in a fixed position to achieve scanning within a 50m range of the track area.

3. Implementation of the OD Algorithm

In this section, we provide a comprehensive description of our algorithm's framework. We begin with the point cloud calibration process and then proceed to explain the rail extraction method using both background point cloud (BP) and foreground point cloud (FP) techniques to achieve OD. Each step will be elaborated on in detail.

The algorithm framework diagram, depicted in Figure 3, illustrates the sequential processing flow.

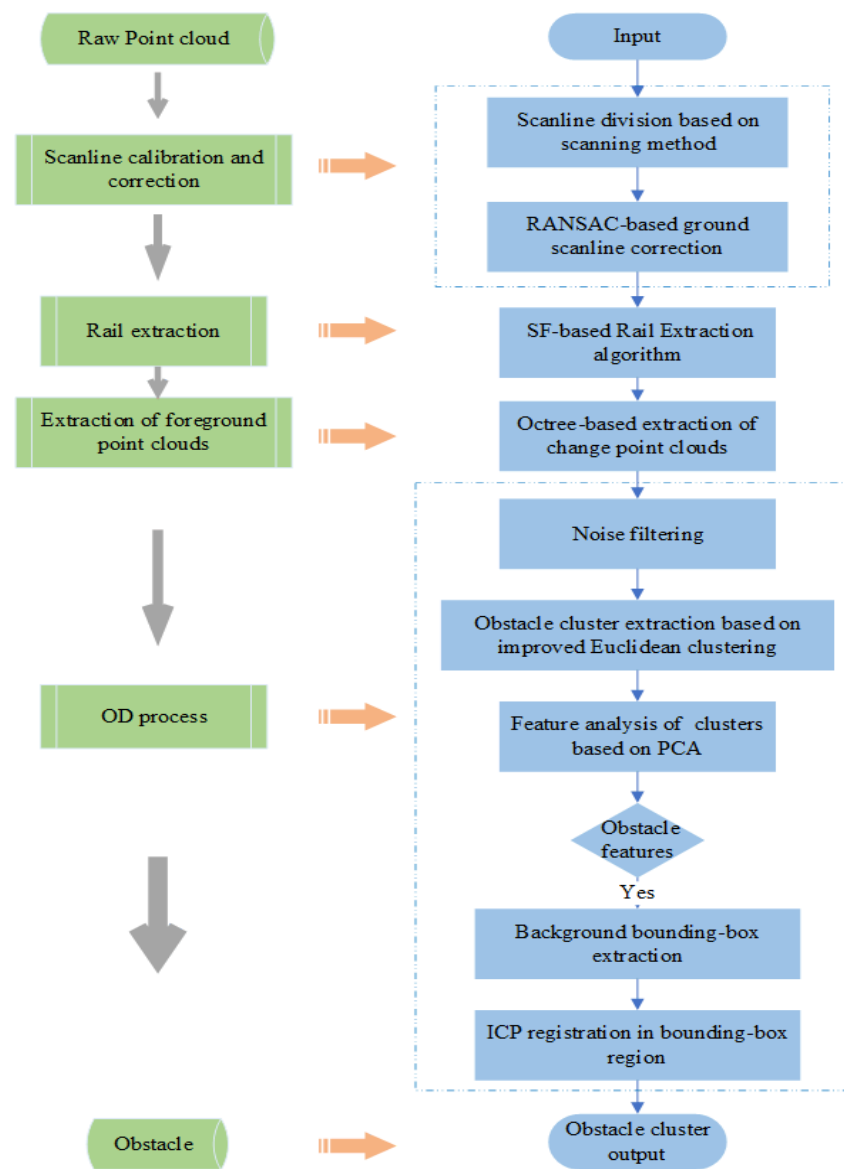


Figure 3. The key steps of our algorithm and its corresponding flowchart.

We have implemented error correction techniques to global scanning lines within a single frame of the point cloud map for both BP and FP. Additionally, we introduce our track extraction algorithm, SFRE (scanline feature-based rail extraction), which leverages the distinct characteristics of scanning lines.

Once the track extraction process is completed, we apply Octree downsampling to the FP, reducing computational complexity. In the primary OD stage, we employ essential algorithms such as Euclidean clustering, PCA (principal component analysis), and local-ICP (local iterative closest point) to perform detailed feature analysis on point cloud clusters. Furthermore, we propose a tunable threshold Euclidean clustering algorithm to address traditional Euclidean methods' limitations when applied to non-uniformly spaced point clouds. This method effectively identifies and outputs point cloud clusters corresponding to obstacles.

3.1. Scanline Calibration and Correction

Our system ensures that point cloud data adhere to a regular single-line distribution, typically containing between 150 k and 200 k points per scene (based on Equation (4)). However, practical applications may introduce errors, such as point cloud drift, due to

installation errors and natural factors like pole sway caused by strong winds. Accurate correction of point cloud errors is crucial as it significantly affects processes like filtering and registration. We first correct the point cloud errors based on the scanlines' distribution characteristics to address this.

We can obtain the transformation formula for coordinates (x, y, z) based on the raw pulse distance R and device height H [27].

$$n = m * T / f \quad (4)$$

$$\begin{cases} x = R * \sin\omega * \cos\theta \\ y = R * \sin\theta \\ z = H - R * \cos\omega * \cos\theta \end{cases} \quad (5)$$

In Equation (4), n represents the total number of points in one frame, m represents the number of scanning lines, T represents the scanning time of a single frame, and f represents the frequency of a single line. In Equation (5), θ represents the horizontal angle of the pulse, ω represents the pitch angle of the pulse, H represents the device's hanging height, and R represents the distance of the pulse.

During the scanning process, the motor's continuous variation results in each pulse point deviating from the ideal ω value. Factors such as shaking and installation introduce errors that accumulate over time, leading to drift in the point cloud of the scene (as depicted in Figure 4). To tackle this issue effectively, we propose a specialized global error correction method.

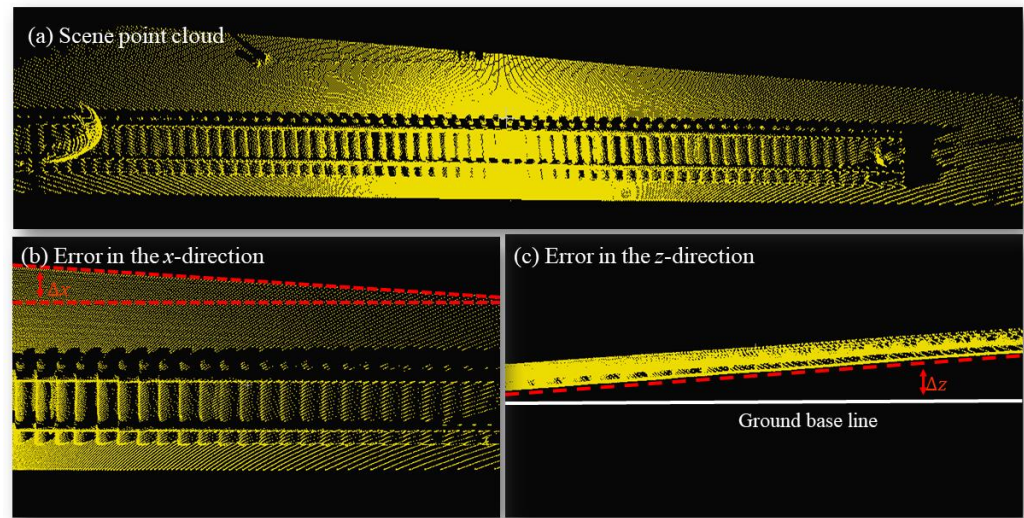


Figure 4. Scene point cloud map and two primary sources of point cloud errors: Δx caused by pitch motion and Δz caused by installation.

The variable $Max \Delta z$ represents the deviation of the scanning lines in the z -direction, which is driven by external factors. The variable $Max \Delta x$ represents the error caused by the same scanning line, which does not follow the same ω value in the x -direction. In our SFRE algorithm, we rely on the x -distribution of the scanning lines. Therefore, we employ the following steps to correct the point cloud (Figure 5):

(a) We input the set of points P from the entire point cloud. Given the known scanning frequency f , we can determine the number of points m in a single scanning line. Using Equation (5), we can accurately segment the input point cloud into individual scanning lines.

$$I_i = \{P_j | P_j \in P_{[m*i, m*(i+1)]}\} \quad (I_i \in I, 0 < i \leq k) \quad (6)$$

where I represents the clustered set of scan lines we have divided and k represents the maximum number of scan lines in a single frame of our point cloud.

(b) We randomly select a baseline scanline, project it onto the xoy plane, and choose a set of scan points. By using the RANSAC algorithm to fit a line [28], we can obtain the spatial parameters of the scanline. Then, we use it to obtain the deviation angle. Similarly, projecting onto the yoz plane gives us the deviation angle. These angles are used to correct the errors caused in Figure 4.

$$ax + by + cz + d = 0 \quad (7)$$

(c) We can obtain the deviation angle based on the fitting parameters of the scanline. We utilize angles α_{xoy} and α_{yoz} to construct the corrective rotation matrix R_1 and R_2 for the scene point cloud representation:

$$R_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\alpha_{xoy}) & -\sin(\alpha_{xoy}) \\ 0 & \sin(\alpha_{xoy}) & \cos(\alpha_{xoy}) \end{bmatrix} \quad (8)$$

$$R_2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\alpha_{yoz}) & -\sin(\alpha_{yoz}) \\ 0 & \sin(\alpha_{yoz}) & \cos(\alpha_{yoz}) \end{bmatrix} \quad (9)$$

$$P' = R_1 R_2 P \quad (10)$$

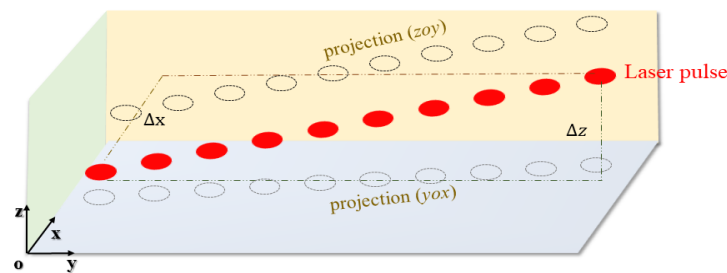


Figure 5. Scanning line projection mechanism in error correction process.

3.2. Railway Extraction

When dealing with large-scale ground point cloud processing, traditional methods for ground segmentation often employ algorithms like RANSAC [29]. However, dealing with the rail track recognition for OD in specific areas of railways, traditional ground segmentation methods encounter significant challenges due to the varying terrains and undulations [30]. Consequently, existing ground segmentation algorithms primarily rely on the assumption of flat ground, resulting in decreased robustness and applicability when confronted with natural terrain variations, including slopes and undulations.

To address this issue, we propose a rail track extraction algorithm based on the scanning characteristics of LiDAR. By analyzing the distribution characteristics of scan lines, our method aims to improve the accuracy and reliability of rail track recognition in the presence of diverse terrains and undulations.

As shown in Figure 6, it is evident that after conducting overall calibration and calibration specifically in the x -direction on the point cloud data, a noticeable “enrichment” phenomenon can be observed in the scan lines during the scanning process from the baseline to the rail track. This implies that the point cloud data yield valuable information in the x -direction during the scanning transformation from the baseline to the rail track. We segmented the scanlines in previous steps and calibrated the point cloud data. Leveraging the features we have discovered, we utilize the enriched region of the x -coordinate as the track boundary for straight-through filtering, enabling us to obtain precise point cloud data about the rail track.

$$p_{rail} \in P (x' \leq p_{rail} \cdot x \leq x'') \quad (11)$$

The pseudo-code for the essential parts is shown as Algorithm 1:

Algorithm 1: Railway scanline division and calibration, SFRE process

Input: point cloud set P , the number of pulses of a scanning line m ;

Output: Rail point cloud P_{rail}

```

1:  $k = 0$ ;
2: for  $i = 1; i < n; i++$  do
3:   Line cluster  $I(k) \leftarrow m$  points
4:    $k++$ 
5: end for
6:  $I(g) \leftarrow$  ground scanning line
7:  $p.z = 0$  ( $p \in I(g)$ )
8:  $\alpha_{xoy} \leftarrow I(g)[50, m - 50]$  RANSAC line fitting
9:  $R_1 \leftarrow \alpha_{xoy}$ 
10:  $p.x = 0$  ( $p \in I(g)$ )
11:  $\alpha_{yoz} \leftarrow I(g)[50, m - 50]$  RANSAC line fitting
12:  $R_2 \leftarrow \alpha_{yoz}$ 
13:  $P' = R_1 R_2 P$ 
14:  $bucket < key, value >$ 
15: for  $i = 0; i < P'.size(); i++$  do
16:   if  $bucket.count(P'.x/A) > 0$  then
17:      $bucket[P'.x/A]++$ 
18:   else  $bucket[P'.x/A] = 1$ 
19:   end if
20: end for
21: //Get the the two keys with the maximum values
22:  $key1, key2 \leftarrow Max(bucket[key1]), Max(bucket[key2])$ 
23:  $P_{rail}.x1 = key1 * A$ 
24:  $P_{rail}.x2 = key2 * A$ 
25: // $f(P, x1, x2)$  filtering in the x direction with set  $P$ 
26:  $Prail \leftarrow f(P', P_{rail}.x1, P_{rail}.x2)$ 
27: return  $Prail$ 

```

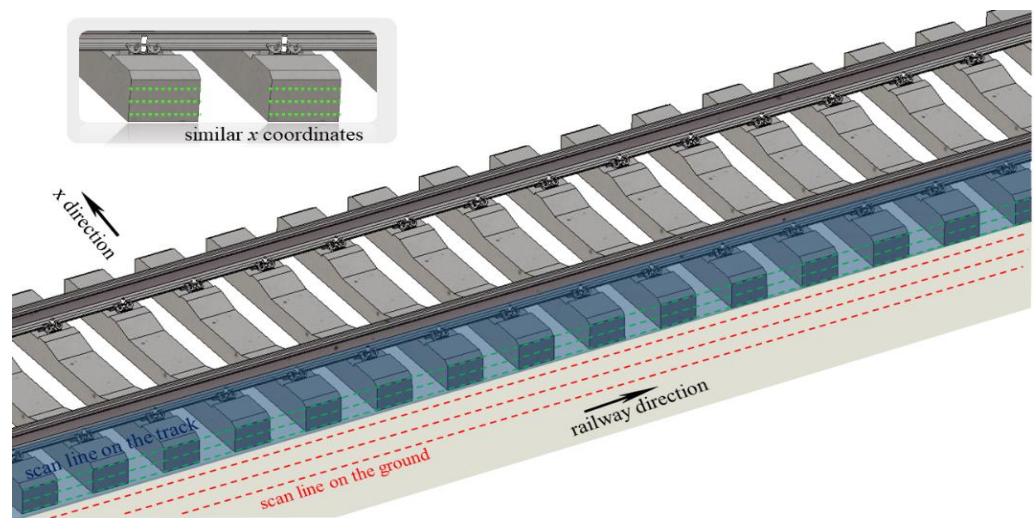


Figure 6. The scanning process of the scanline on the railway track.

3.3. OD Process

Before entering the OD phase, we utilized Octree to reduce computational complexity and minimize computational overhead [31]. We downsampled using a fixed step size ρ , obtaining sparse samples of the original point cloud for subsequent calculations.

3.3.1. Improved Euclidean Clustering

The traditional Euclidean clustering algorithm is implemented through the following steps: Select a center point q_1 , search for n nearest points within a specified threshold value, and create a set P that satisfies the threshold. Next, select another point q_2 from set P and repeat the process until a complete set P is formed. The traditional Euclidean clustering algorithm [32] uses a fixed threshold value throughout the process. However, in practical applications (as shown in Figure 7), the point cloud density varies significantly with distance R along horizontal and vertical scanning directions. This variation reduces the adaptability of the traditional Euclidean clustering algorithm. To address this issue, we propose a tunable threshold Euclidean clustering method.

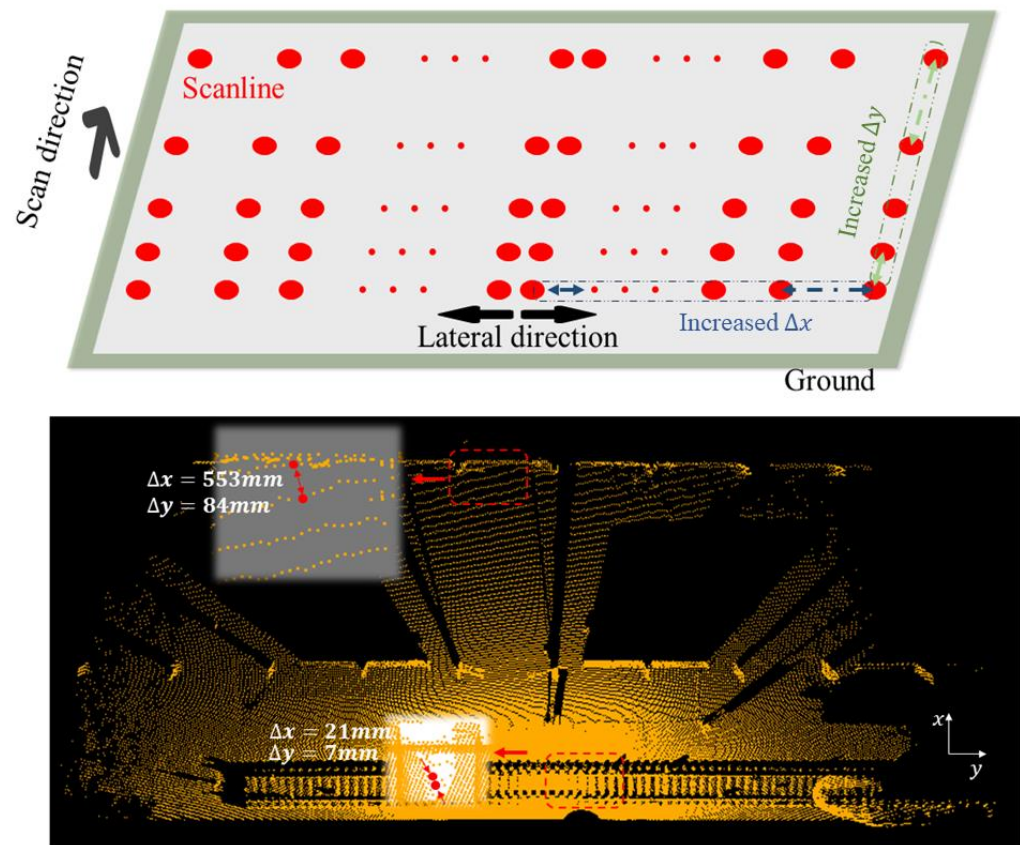


Figure 7. The distribution trend of point cloud density in the x - and y -directions within the actual scanning scene, where Δx represents the distance between the x -directions of two points on the same plane and Δy represents the distance between the y -directions of two points [33].

We observed that the region with the highest point cloud density occurs in the central area of the scanning line at the starting pitch angle ω . Therefore, we propose the following tunable threshold strategy:

$$\varepsilon_{adaptive} = f(x, y, p_i) \varepsilon \quad (12)$$

In this equation, the variable ε represents the initial input threshold in traditional Euclidean clustering algorithms. Additionally, $f(x, y, p_i)$ signifies the modulation

function associated with the x - and y -directions of the clustering center point p_i . The detailed form of the modulation function can be found in Section 4.

3.3.2. PCA Process for Clusters

After obtaining different point cloud clusters using the improved Euclidean clustering method, we performed principal component analysis (PCA) [34] on the point cloud to obtain feature information of obstacle clusters O for filtering and differentiation. The PCA process is as follows [35]: we input the obstacle clusters according to the index and obtain the number of points in the i -th cluster, denoted as O_i . If the information in the point cloud is (x, y, z) , we obtain an $n \times 3$ point cloud matrix ξ . We perform mean normalization on the data of each point in matrix ξ , and then obtain the covariance matrix C ; using Equation (16), we obtain the three eigenvectors $E(e_1, e_2, e_3)$ and eigenvalues $(\lambda_1, \lambda_2, \lambda_3)$, where Λ is a diagonal matrix.

$$O_s = \frac{1}{n} \sum_{i=1}^n p_i \quad (p_i \in \xi) \quad (13)$$

$$p_i = p_i - O_s \quad (i \in [1, n]) \quad (14)$$

$$C = \frac{1}{n} \xi \xi^T \quad (15)$$

$$E^T C E = \Lambda \quad (16)$$

We arrange the eigenvalues in descending order as $\lambda_1 > \lambda_2 > \lambda_3$. According to the relationship between eigenvalues, we make the following division of cluster types: line type, plane type, and unknown type (Figure 8).

$$\begin{cases} \lambda_1/\lambda_2 > \mu_1 & \text{lines} \\ \text{else } \lambda_2/\lambda_3 > \mu_2 & \text{plane} \\ & \text{unknown} \end{cases} \quad (17)$$

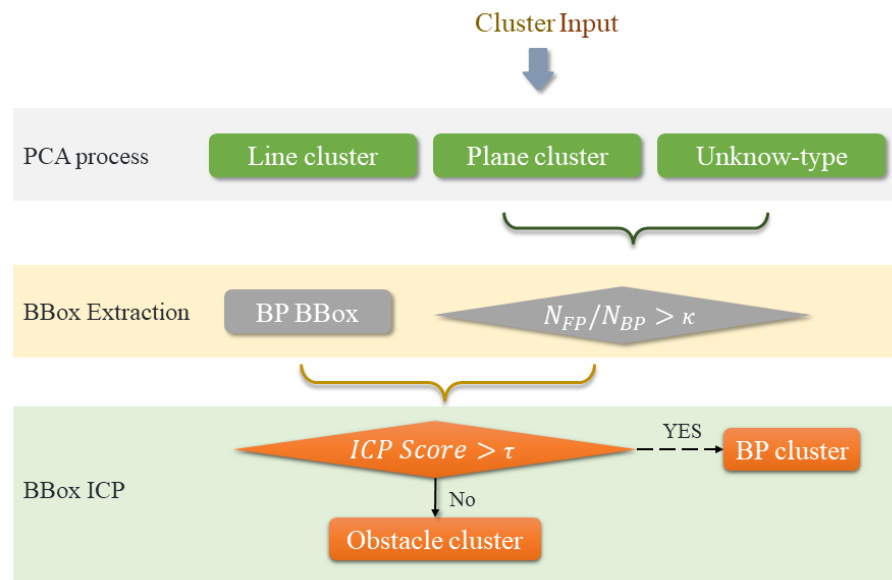


Figure 8. The mechanisms and procedures of PCA and local-ICP processing in the OD process.

We obtained a collection of multiple clusters, denoted as T_1, T_2 and T_3 . Where T_1 represents a cluster collection of lines point clouds, T_2 represents a cluster collection of plane point clouds, and T_3 represents a cluster collection of unknown point clouds. By default, we only remove cluster T_1 . For cluster T_2 , sometimes due to the perspective, we

can only see one side of the object. Therefore, it is treated as a potential obstacle cluster. We extract equal-sized regions from the background point cloud based on the indices of all clusters. When Formula (18) is satisfied, we consider that the cluster is more likely to be an obstacle.

$$N_{FP}/N_{BP} > \kappa \quad (18)$$

Here, N_{BP} represents the number of point clouds within the bounding box belonging to BP, N_{FP} represents the number of point clouds within the bounding box belonging to FP, and κ is our parameter threshold.

3.3.3. Local-ICP Process for Clusters

The ICP process is commonly used for point cloud registration, aiming to find the best correspondence between a source point cloud and a target point cloud through iterative refinement [36]. Once we obtain the bounding boxes of the BP (BP BBox) and FP (FP BBox), we can achieve the alignment of the two through the following steps using local-ICP [34]:

- Use the BP BBox point cloud as the source input point cloud set P and the FP BBox point cloud as the target point cloud set Q .
- Find the correspondence point q_i for each p_i .
- Use an energy minimization strategy to find the optimal transformation matrix $T(R, t)$ that satisfies Equation (19).
- Repeat the iteration process until step (c) converges to meet the threshold.

$$f(R, t) = \min_{R, t} \sum_{i=1}^n \|Rp_i + t - q_i\|^2 \quad (19)$$

After registration using local-ICP, when the registration score is high, the probability of the cluster not belonging to obstacles is high. Taking the complement of the cluster set results in the obstacle set.

$$\tau = \frac{\sum_{i=1}^n \|p_i - q_i\|^2}{n} \quad (20)$$

4. Experiment

To evaluate the effectiveness of our proposed algorithm, we conducted a quantitative analysis by creating a standardized simulated railway track measuring 50 m in length. The algorithm was subsequently deployed in our experimental site, and this section provides a detailed demonstration of our algorithm's strategy.

4.1. Evaluation of Error Correction

We refer to Table 1 for the hardware parameters of the equipment used. In order to address the errors mentioned in Section 2, we partitioned the scan lines based on a fixed number of points per scanline after obtaining the complete point cloud of a single frame scene. For calibration of these two types of errors, we applied Formula (21):

$$\begin{cases} \Delta x = p_i \cdot x - p_j \cdot x \\ \Delta z = p_i \cdot z - p_j \cdot z \end{cases} (p_i, p_j \in p \subset I) \quad (21)$$

Table 1. Hardware parameters of the equipment.

Variable	Definition	Value
f	Single line frequency	50 (Hz)
m	Number of points in a single scan line	511
T	Scan time	6 (s)
H	Device height	3200 (mm)

δ_{pitch}	Pitch angle resolution	0.13(°)
$\delta_{horizontal}$	Horizontal angular resolution	0.33(°)
k	Scan line number	300

The variables p_i and p_j represent the i -th and j -th scanned points in the scanline p (belonging to scan line clusters I). Given the high probability of noise near the edges of scan lines and their high variability, we measured x and z using $i = 60$ and $j = 350$. We statistically analyzed the maximum error of all scan lines in random scenes (Figure 9), revealing that the maximum values of x and z were 467 mm and 809 mm. This result confirms the effectiveness of our correction strategy. Furthermore, to obtain point cloud information with accurate coordinates for subsequent algorithmic steps, we calibrated the point cloud data using the fitted matrix R_1 and R_2 .

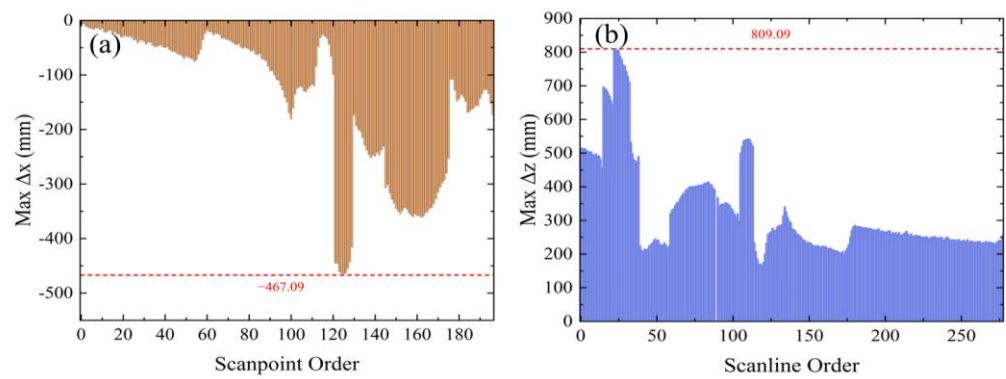


Figure 9. (a) Maximum statistical error of Δx in random scene point clouds; (b) maximum statistical error of Δz in random scene point clouds.

4.2. Evaluation of SFRE Method

We analyzed the x -coordinate of the scan line set I , as depicted in Figure 10. During the scanning process from the ground base to the track, we observed a notable clustering of x -coordinates within a narrow range of variations. This trend is illustrated by the pattern shown in Figure 11. Leveraging this observation, we can identify abrupt changes in the x -coordinate as indicative of the boundary regions of the track. More specifically, when the condition specified by Formula (22) is satisfied, it indicates the presence of an edge region in the track.

$$\Delta m / \Delta p.x > A \quad (p \in I) \quad (22)$$

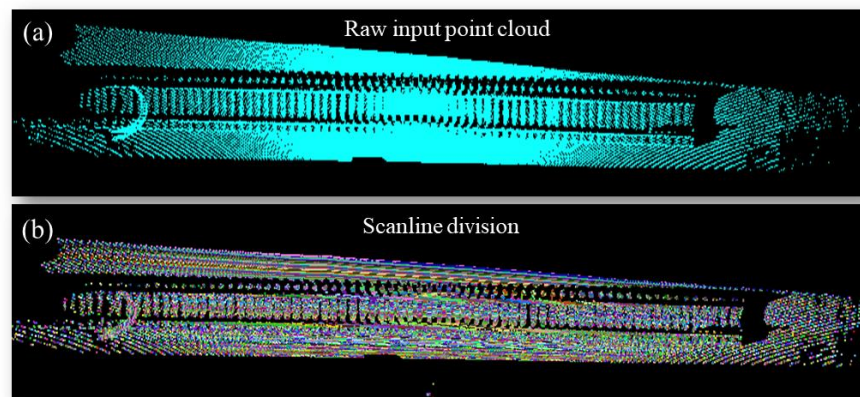


Figure 10. (a) Raw input point cloud; (b) scanline division process, and we used the same color for each scanning line.

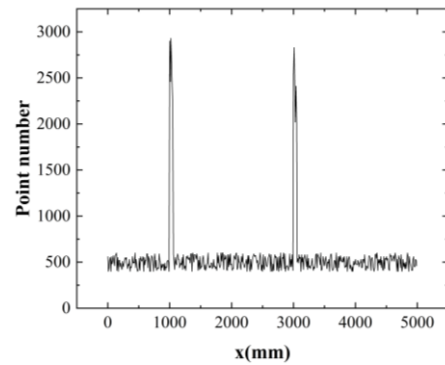


Figure 11. Statistical analysis of the number of scan points at the same x position obtained using Formula (22).

Here, I represents the set of clusters of scanlines and p is a subset of this set. The formula indicates that the region with a sudden change in the pulse count corresponds to the edge region of the track, as the x -coordinate between scan lines changes.

We compared our developed track extraction algorithm, the traditional RANSAC ground segmentation, and the region growing algorithm based on the point cloud library (PCL). It is essential to strike a balance in segmentation, as excessive segmentation can lead to losing track features within the detected region. In contrast, insufficient segmentation may result in point cloud outliers being mistakenly classified as noise. To evaluate their performance, we meticulously selected random scenes and measured the runtime of each algorithm, as well as the retention rate of valid points.

$$RR = \frac{N(P_{rail})}{N(P)} \quad (23)$$

where $N(P_{rail})$ refers to the number of point clouds in the segmented track area, while $N(P)$ refers to the total number of point clouds.

As depicted in Figure 12 after adjusting to the optimal parameters, both the RANSAC algorithm and the region growing algorithm exhibited varying degrees of over-segmentation and under-segmentation, with non-ground clusters unable to be fully separated. Our algorithm showcases remarkable robustness when handling minor terrain variations or small-scale, irregularly shaped objects on the ground. It can accurately extract the track while effectively filtering out interfering point clouds from non-detection areas. Furthermore, in Figure 13, our algorithm demonstrates high computational efficiency in terms of runtime, enabling the preservation of a more significant number of feature point clouds in the track region. This facilitates seamless progress in subsequent analysis steps.

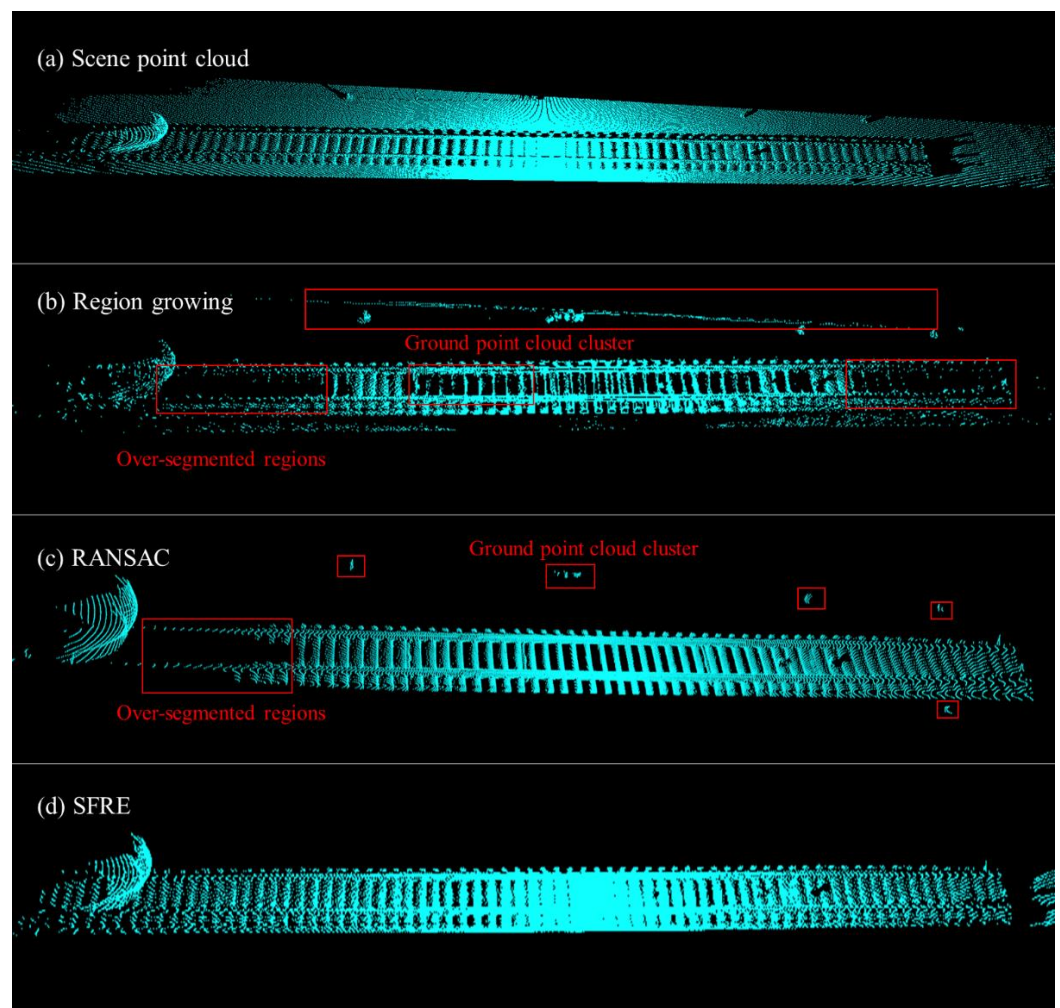


Figure 12. A comparison of the segmentation and extraction results from three algorithms: (a) scene point cloud; (b) ground segmentation using the region growing method (neighbor point: 10; smoothing threshold: 3 rad; curvature threshold: 1 rad); (c) ground segmentation using RANSAC (iterations: 1500; distance threshold: 85 mm); (d) track extraction result using our SFRE algorithm.

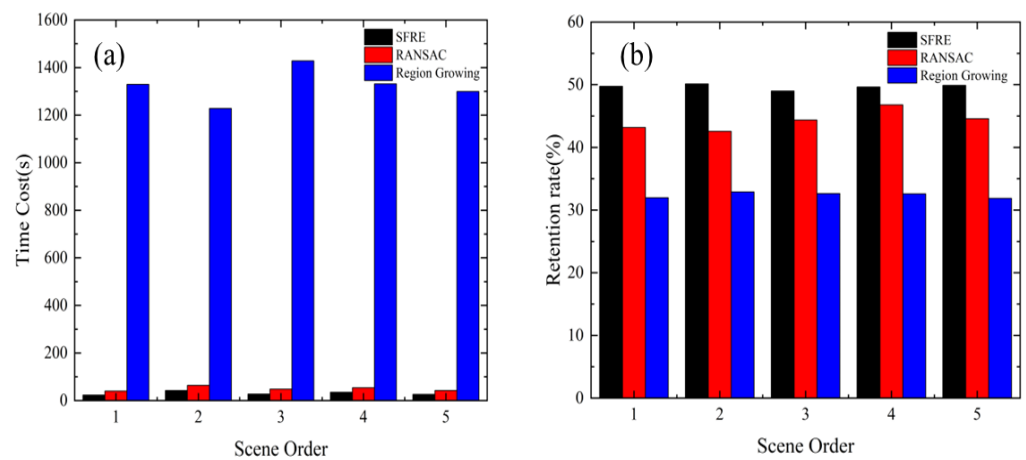


Figure 13. A comparison of three algorithms: RANSAC ground segmentation, region growing segmentation, and SFRE. The figure presents two key aspects: (a) shows the computational time required by each algorithm, and (b) shows the retention rate of valid points.

4.3. Evaluation of Improved Euclidean Clustering

We conducted a quantitative analysis of the distribution difference in the x - and y -directions based on the scanning characteristics of the device.

$$\begin{cases} x_i = \frac{H}{\cos \omega} * \tan|\theta_i| \ (0 \leq \theta_i < \frac{\pi}{2}) \\ \theta_i = i * \delta_{horizontal} \ \{i|0 \leq i < \frac{m}{2}\} \\ \Delta x = x_{i+1} - x_i \end{cases} \quad (24)$$

$$\begin{cases} y_j = H * \tan\omega_j \ (\omega_0 \leq \omega_j \leq \omega_t) \\ \omega_j = j * \delta_{pitch} \ \{j|1 \leq j \leq k\} \\ \Delta y = y_{j+1} - y_j \end{cases} \quad (25)$$

Using Equations (24) and (25), we have chosen the parameters Δx and Δy to represent the density distribution trends of the point cloud data along the x and y directions. Our aim is to ensure that these variables accurately represent the graphical representation of the point cloud's density distribution (the results are shown in Figure 14).

$$f(x, y) = e^{\frac{|p_i \cdot x|}{x_{max}} + \frac{|p_i \cdot y|}{y_{max}} + 1} \quad (26)$$

Considering the exponential distribution of spacing differences in the x - and y -directions and aiming to limit threshold divergence, we have employed Formula (26) as a modulation function. In this formula, $p_i \cdot x$ represents the x -coordinate of the iterative center point p_i , $p_i \cdot y$ represents the y -coordinate of the center point p_i , and x_{max} and y_{max} represent the boundary values of x and y after removing outliers. By dynamically adjusting the clustering centroid points p_i , our algorithm achieves tunable threshold control in both the x and y directions.

We conducted extensive experimental scenarios to evaluate our modulation function's effectiveness. The results demonstrated that suspicious clusters were effectively detected even in distant areas, minimizing the risk of missing small objects at far distances (as shown in Figure 15). Moreover, it significantly reduced the occurrence of secondary splitting within the same cluster due to distance divergence caused by LiDAR at far distances. This exemplifies the high detection rate of our algorithm, particularly for high-precision objects.

Overall, our modulation function enables precise and adaptable threshold control, ensuring reliable object detection and reducing the impact of distance divergence on clustering outcomes.

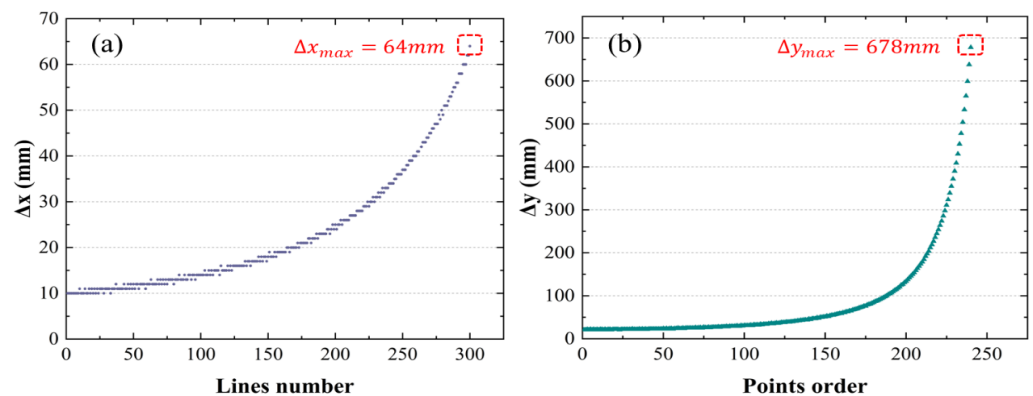


Figure 14. Statistical analysis of Δx and Δy in the x - and y -directions, calculated using Formulas (24) and (25).

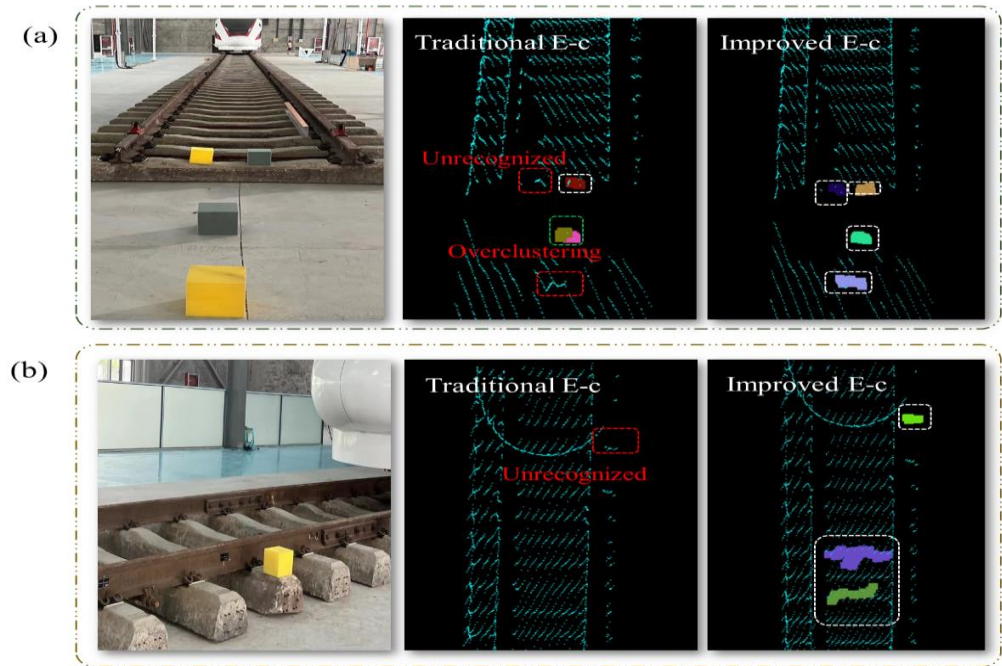


Figure 15. Application analysis of traditional Euclidean algorithm and improved Euclidean algorithm for long-range distances: (a) clustering implementation in multi-box scenario ($y = -2500$ mm); (b) clustering effect for small objects at long distances ($y = 2000$ mm).

4.4. OD Process Based on PCA and Local-ICP

After identifying a cluster of suspicious obstacles O , which may contain numerous false positives ($O_{line} \cup O_{plane} \cup O_{else} = O$). We applied PCA to the clusters to extract a subset that excludes the false obstacle clusters O_{line} . This step allowed us to categorize the point cloud clusters into plane and unknown point clouds. Subsequently, we extracted the point clouds in the BP BBox for these clusters and performed sequential registration using local-ICP. To further refine the results, we employed Formula (19) to enhance the filtering of the obstacle set, resulting in $O_{obstacles}$ ($O_{obstacles} \subseteq (O_{plane} \cup O_{else})$). Throughout this process, we fine-tuned the parameters and conducted experiments to optimize the outcome.

To evaluate the performance of our algorithm, we conducted performance tests on the experimental site using the parameters listed in Table 2. The stable results obtained from these tests are documented in Table 3, where we primarily utilized the SIDR (single detection rate) and STDR (stable detection rate) as our detection metrics.

$$SIDR = \frac{S_{single}}{S} \quad (27)$$

$$STDR = \frac{S_{stable}}{S} \quad (28)$$

where S represents the total number of samples, S_{single} represents the number of samples detected in a single instance, and S_{stable} represents the number of samples detected more than once.

Extensive test results have demonstrated that our algorithm exhibits sufficient recognition capability for obstacles of size $15 \text{ cm} \times 15 \text{ cm} \times 15 \text{ cm}$ at 25 m on both sides of the track. Additionally, it also demonstrates the ability to recognize obstacles of size $10 \text{ cm} \times 10 \text{ cm} \times 10 \text{ cm}$ at 20 m on both sides (Figure 16 is a step-by-step diagram of our algorithm pipeline running). We have also compared the indicators of similar algorithms that have been reported and listed them in Table 4. Currently, our algorithm has shown better

performance in terms of its detection size with the obstacles, tolerance for obstacle size, and detection stability.

In addition, during the process of train transportation, some threatening obstacles often appear in the form of irregular obstacles. To further demonstrate the detection effect of our algorithm on irregular obstacles, we tested the algorithm on large-sized stones, pedestrians, and other obstacles that may affect train operation. We adopted the same testing parameters in Table 2 and obtained the algorithm running results as shown in Figure 17. The results show that our algorithm has successfully detected all possible sample obstacles at present and that our system provides a new solution for railway safety.

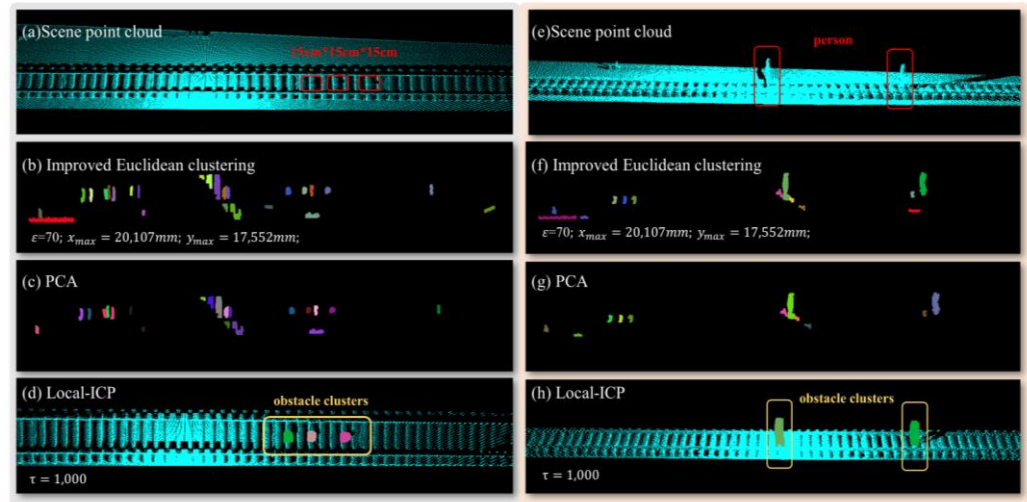


Figure 16. The main process and diagrams of the OD algorithm: (a) multiple box obstacles with dimensions 15 cm × 15 cm × 15 cm; (b) cluster O after the improved Euclidean algorithm is applied ($\varepsilon = 70$; $x_{max} = 20,107$ mm; $y_{max} = 17,552$ mm); (c) the collection of obstacle clusters obtained after performing PCA; (d) the correct cluster $O_{obstacles}$ achieved by using the local-ICP algorithm ($\tau = 1000$); (e–h) a comparable processing procedure for obstacles represented as human entities.

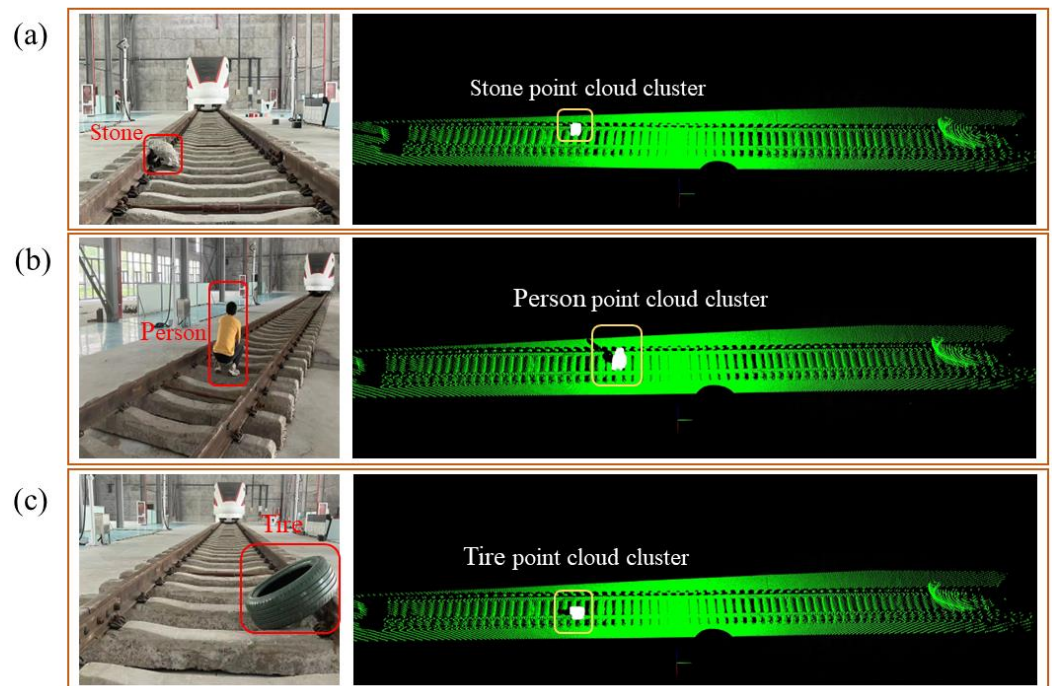


Figure 17. We additionally placed three types of irregular obstacles and displayed on-site photos and algorithm operation results: (a) an irregular stone; (b) A person located in the track area; (c) a tire located in the track area. All types of obstacles have been successfully detected.

Table 2. Parameter values of the key steps in our algorithm workflow.

ρ	ϵ	μ_1	μ_2	κ	τ
100	70	100	10	1.1	1000

Table 3. After conducting a substantial number of sample experiments, we evaluated the performance of our algorithm using the SIDR and STDR metrics.

Obstacles	Size (cm)	Detection Distance (m)							
		0		10		20		25	
		SIDR	STDR	SIDR	STDR	SIDR	STDR	SIDR	STDR
person	50 × 50 × 175	100%	100%	100%	100%	100%	100%	100%	100%
box	20 × 20 × 20	100%	100%	100%	100%	100%	100%	100%	100%
box	15 × 15 × 15	100%	100%	100%	100%	95%	100%	92%	96%
box	10 × 10 × 10	93%	97%	86%	94%	72%	84%	-	-

Table 4. Comparison of metrics between our model and similar algorithms.

Model	Min Detection Size (cm)	Max Range of Action (m)	Stable Detection Rate (%)
Vitor [37]	30	10	-
J. Li [38]	10	<15	-
G. Zhu [39]	10	20	>70
Ours	10	20	>84

5. Conclusions

The application of LiDAR in railway OD holds significant research potential. Using the 3D mechanical LiDAR, we have developed an innovative algorithm for obstacle detection based on track area point clouds.

Firstly, we analyzed and corrected the sources of error causing overall misalignment in the point cloud data. By calibrating the point cloud data, we achieved improved accuracy. Instead of relying on traditional ground segmentation algorithms, we employed an SFRE algorithm that retains the track characteristics more effectively. Additionally, we applied Octree downsampling to reduce computational overhead.

During the Euclidean clustering process, we encountered limitations with fixed-threshold applications. To address this, we introduced a modulation function based on the distribution of point cloud density, enabling adaptive neighborhood adjustment. The results demonstrated the effectiveness of our clustering approach, even when dealing with reduced point cloud density.

Following extracting Euclidean clusters from the point cloud data, we implemented PCA to identify potential obstacle sets. Using local-ICP, we filtered out false positive clusters that exhibited significant differences compared to the background point clouds, accurately identifying obstacle point clouds. To ensure robustness, we constructed a standardized railway simulation site to test and optimize our algorithm continuously. The results demonstrated stable detection capability for 10 cm × 10 cm × 10 cm obstacles at 20 m on both sides of the track, with accuracy thoroughly evaluated.

In future efforts, we will further optimize our algorithm to ensure its robust operation under challenging weather conditions such as heavy fog and intense rainfall.

Author Contributions: Conceptualization, Z.N. and G.Z.; methodology, Z.N.; writing—original draft preparation, X.Z.; writing—review and editing, X.Z. and X.L.; supervision, X.L.; funding

acquisition, Y.Y. All authors have read and agreed to the published version of the manuscript. All authors have read and agreed to the published version of the manuscript.

Funding: This work was financially supported by the CAS Project for Young Scientists in Basic Research (Grant No. YSBR-065), the National Natural Science Foundation of China (No. 62275244, No. 62225507, No. U2033211, No. 62175230, No. 62175232), the Scientific Instrument Developing Project of the Chinese Academy of Sciences (Grant No. YJKYYQ20200001), the National Key R&D Program of China (No. 2022YFB3607800, No. 2022YFB3605800, No. 2022YFB4601501), and the Key Program of the Chinese Academy of Sciences (ZDBS-ZRKJZ-TLC018).

Data Availability Statement: The authors confirm that the data supporting the findings of this study are available within the article.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Zhang, Q.; Yan, F.; Song, W.; Wang, R.; Li, G. Automatic Obstacle Detection Method for the Train Based on Deep Learning. *Sustainability* **2023**, *15*, 1184. <https://doi.org/10.3390/su15021184>.
2. Feng, J.; Liu, L.; Pei, Q.; Li, K. Min-Max Cost Optimization for Efficient Hierarchical Federated Learning in Wireless Edge Networks. *IEEE Trans. Parallel Distrib. Syst.* **2022**, *33*, 2687–2700. <https://doi.org/10.1109/tpds.2021.3131654>.
3. Qu, J.; Li, S.; Li, Y.; Liu, L. Research on Railway Obstacle Detection Method Based on Developed Euclidean Clustering. *Electronics* **2023**, *12*, 1175. <https://doi.org/10.3390/electronics12051175>.
4. Li, Y.; Zhu, L.; Wang, H.; Yu, F.R.; Liu, S. A Cross-Layer Defense Scheme for Edge Intelligence-Enabled CBTC Systems Against MitM Attacks. *IEEE Trans. Intell. Transp. Syst.* **2021**, *22*, 2286–2298. <https://doi.org/10.1109/tits.2020.3030496>.
5. Wang, J.; Xu, M.; Foroughi, F.; Dai, D.; Chen, Z. FasterGICP: Acceptance-Rejection Sampling Based 3D Lidar Odometry. *IEEE Robot. Autom. Lett.* **2022**, *7*, 255–262. <https://doi.org/10.1109/lra.2021.3124072>.
6. Miao, Y.; Tang, Y.; Alzahrani, B.A.; Barnawi, A.; Alafif, T.; Hu, L. Airborne LiDAR Assisted Obstacle Recognition and Intrusion Detection Towards Unmanned Aerial Vehicle: Architecture, Modeling and Evaluation. *IEEE Trans. Intell. Transp. Syst.* **2021**, *22*, 4531–4540. <https://doi.org/10.1109/tits.2020.3023189>.
7. Grollius, S.; Buchner, A.; Ligges, M.; Grabmaier, A. Probability of Unrecognized LiDAR Interference for TCSPC LiDAR. *IEEE Sens. J.* **2022**, *22*, 12976–12986. <https://doi.org/10.1109/jsen.2022.3178179>.
8. Zhu, G. et al. High anti-interference 3D imaging LIDAR system based on digital chaotic pulse position modulation. *Opt. Laser Technol.* **2023**, *163*, 109405. <https://doi.org/10.1016/j.optlastec.2023.109405>.
9. Charles, R.Q.; Su, H.; Kaichun, M.; Guibas, L.J. PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation. In Proceedings of the 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Honolulu, HI, USA, 21–26 July 2017.
10. Qi, C.R.; Yi, L.; Su, H.; Guibas, L.J. PointNet plus plus: Deep Hierarchical Feature Learning on Point Sets in a Metric Space. In Proceedings of the Advances in Neural Information Processing Systems 30 (NIPS 2017), Long Beach, CA, USA, 4–9 December 2017.
11. Zhou, C.; Li, F.; Cao, W.; Wang, C.; Wu, Y. Design and implementation of a novel obstacle avoidance scheme based on combination of CNN-based deep learning method and LiDAR-based image processing approach. *J. Intell. Fuzzy Syst.* **2018**, *35*, 1695–1705. <https://doi.org/10.3233/jifs-169706>.
12. Jiang, W.; Chen, W.; Song, C.; Yan, Y.; Zhang, Y.; Wang, S. Obstacle detection and tracking for intelligent agricultural machinery. *Comput. Electr. Eng.* **2023**, *108*, 108670. <https://doi.org/10.1016/j.compeleceng.2023.108670>.
13. Sun, Y.; Zuo, W.; Huang, H.; Cai, P.; Liu, M. PointMoSeg: Sparse Tensor-Based End-to-End Moving-Obstacle Segmentation in 3-D Lidar Point Clouds for Autonomous Driving. *IEEE Robot. Autom. Lett.* **2021**, *6*, 510–517. <https://doi.org/10.1109/lra.2020.3047783>.
14. Hata, A.Y.; Osorio, F.S.; Wolf, D.F. Robust curb detection and vehicle localization in urban environments. In Proceedings of the 2014 IEEE Intelligent Vehicles Symposium Proceedings, Ypsilanti, MI, USA, 8–11 June 2014; pp. 1257–1262. <https://doi.org/10.1109/IVS.2014.6856405>.
15. Asvadi, A.; Premebida, C.; Peixoto, P.; Nunes, U. 3D Lidar-based static and moving obstacle detection in driving environments: An approach based on voxels and multi-region ground planes. *Robot. Auton. Syst.* **2016**, *83*, 299–311. <https://doi.org/10.1016/j.robot.2016.06.007>.
16. Miao, Y.; Li, S.; Wang, L.; Li, H.; Qiu, R.; Zhang, M. A single plant segmentation method of maize point cloud based on Euclidean clustering and K-means clustering. *Comput. Electron. Agric.* **2023**, *210*, 107951. <https://doi.org/10.1016/j.compag.2023.107951>.
17. Guo, Z.; Liu, H.; Shi, H.; Li, F.; Guo, X.; Cheng, B. KD-Tree-Based Euclidean Clustering for Tomographic SAR Point Cloud Extraction and Segmentation. *IEEE Geosci. Remote Sens. Lett.* **2023**, *20*, 1–5. <https://doi.org/10.1109/lgrs.2023.3234406>.
18. Walicka, A.; Pfeifer, N. Automatic Segmentation of Individual Grains from a Terrestrial Laser Scanning Point Cloud of a Mountain River Bed. *IEEE J. Sel. Top. Appl. Earth Obs. Remote Sens.* **2022**, *15*, 1389–1410. <https://doi.org/10.1109/JSTARS.2022.3141892>.
19. Gao, F.; Li, C.; Zhang, B. A Dynamic Clustering Algorithm for Lidar Obstacle Detection of Autonomous Driving System. *IEEE Sens. J.* **2021**, *21*, 25922–25930. <https://doi.org/10.1109/jsen.2021.3118365>.
20. Jiang, W.; Song, C.; Wang, H.; Yu, M.; Yan, Y. Obstacle Detection by Autonomous Vehicles: An Adaptive Neighborhood Search Radius Clustering Approach. *Machines* **2023**, *11*, 54. <https://doi.org/10.3390/machines11010054>.

21. Xie, D.; Xu, Y.; Wang, R. Obstacle detection and tracking method for autonomous vehicle based on three-dimensional LiDAR. *Int. J. Adv. Robot. Syst.* **2019**, *16*, 172988141983158. <https://doi.org/10.1177/1729881419831587>.
22. Moosmann, F.; Stiller, C. Joint self-localization and tracking of generic objects in 3D range data. In Proceedings of the 2013 IEEE International Conference on Robotics and Automation, Karlsruhe, Germany, 6–10 May 2013; pp. 1146–1152. <https://doi.org/10.1109/ICRA.2013.6630716>.
23. Che, E.; Jung, J.; Olsen, M.J. Object Recognition, Segmentation, and Classification of Mobile Laser Scanning Point Clouds: A State of the Art Review. *Sensors* **2019**, *19*, 810. <https://doi.org/10.3390/s19040810>.
24. Borgmann, B.; Hebel, M.; Arens, M.; Stilla, U. Detection of Persons in MLS Point Clouds Using Implicit Shape Models. *Int. Arch. Photogramm. Remote Sens. Spat. Inf. Sci.* **2017**, *XLII-2/W7*, 203–210.
25. Burger, P.; Wuensche, H.J. Fast Multi-Pass 3D Point Segmentation Based on a Structured Mesh Graph for Ground Vehicles. In Proceedings of the 2018 IEEE Intelligent Vehicles Symposium (IV), Changshu, China, 26–30 June 2018; pp. 2150–2156. <https://doi.org/10.1109/IVS.2018.8500552>.
26. Wang, G.; Wu, J.; He, R.; Yang, S. A Point Cloud-Based Robust Road Curb Detection and Tracking Method. *IEEE Access* **2019**, *7*, 24611–24625. <https://doi.org/10.1109/access.2019.2898689>.
27. Xu, X.; Zhao, M.; Lu, Y.; Ran, Y.; Tan, Z.; Luo, M. Design of 2D LiDAR and camera fusion system improved by differential evolutionary PID with nonlinear tracking compensator. *Infrared Phys. Technol.* **2021**, *116*, 103776. <https://doi.org/10.1016/j.infrared.2021.103776>.
28. Dong, Y.; Liang, C.; Sun, Z. An Improved Phase Correlation Subpixel Remote Sensing Registration Algorithm Using Probability-Guided RANSAC. *IEEE Geosci. Remote Sens. Lett.* **2022**, *19*, 1–5. <https://doi.org/10.1109/lgrs.2022.3183636>.
29. Zou, W.; Shen, D.; Cao, P.; Lin, C.; Zhu, J. Fast Positioning Method of Truck Compartment Based on Plane Segmentation. *IEEE J. Radio Freq. Identif.* **2022**, *6*, 774–778. <https://doi.org/10.1109/jrfid.2022.3213852>.
30. Anand, B.; Senapati, M.; Barsaiyan, V.; Rajalakshmi, P. LiDAR-INS/GNSS-Based Real-Time Ground Removal, Segmentation, and Georeferencing Framework for Smart Transportation. *IEEE Trans. Instrum. Meas.* **2021**, *70*, 8504611. <https://doi.org/10.1109/tim.2021.3117661>.
31. Li, L.; Li, Z.; Liu, S.; Li, H. Motion Estimation and Coding Structure for Inter-Prediction of LiDAR Point Cloud Geometry. *IEEE Trans. Multimed.* **2022**, *24*, 4504–4513. <https://doi.org/10.1109/tmm.2021.3119872>.
32. Cao, Y.; Wang, Y.; Xue, Y.; Zhang, H.; Lao, Y. FEC: Fast Euclidean Clustering for Point Cloud Segmentation. *Drones* **2022**, *6*, 325. <https://doi.org/10.3390/drones6110325>.
33. Warchoń, A.; Karaś, T.; Antoń, M. Selected Qualitative Aspects of Lidar Point Clouds: Geoslam Zeb-Revo and Faro Focus 3D X130. *Int. Arch. Photogramm. Remote Sens. Spatial Inf. Sci.* **2023**, *XLVIII-1/W3*, 205–212. <https://doi.org/10.5194/isprs-archives-XLVIII-1-W3-2023-205-2023>.
34. Hu, L.; Xiao, J.; Wang, Y. An automatic 3D registration method for rock mass point clouds based on plane detection and polygon matching. *Vis. Comput.* **2019**, *36*, 669–681. <https://doi.org/10.1007/s00371-019-01648-z>.
35. Duan, Y.; Yang, C.; Chen, H.; Yan, W.; Li, H. Low-complexity point cloud denoising for LiDAR by PCA-based dimension reduction. *Opt. Commun.* **2021**, *482*, 126567. <https://doi.org/10.1016/j.optcom.2020.126567>.
36. Yilmaz, A.; Temeltas, H. Robust affine registration method using line/surface normals and correntropy criterion. *Complex Intell. Syst.* **2022**, *8*, 1–19. <https://doi.org/10.1007/s40747-021-00599-0>.
37. Amaral, V.; Marques, F.; Lourenço, A.; Barata, J.; Santana, P. Laser-Based Obstacle Detection at Railway Level Crossings. *J. Sens.* **2016**, *2016*, 1–11. <https://doi.org/10.1155/2016/1719230>.
38. Li, J.; Li, R.; Wang, J.Z.; Yan, M. Obstacle information detection method based on multiframe three-dimensional lidar point cloud fusion. *Opt. Eng.* **2019**, *58*, 116102. <https://doi.org/10.1117/1.OE.58.11.116102>.
39. Zhu, G.; Nan, Z.; Zhang, X.; Yang, Y.; Liu, X.; Lin, X. High precision rail surface obstacle detection algorithm based on 3D imaging LiDAR. *Opt. Lasers Eng.* **2024**, *178*, 108206. <https://doi.org/10.1016/j.optlaseng.2024.108206>.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.